



**Министерство науки и высшего образования Российской Федерации**  
**Федеральное государственное бюджетное образовательное учреждение**  
**высшего образования**  
**«Московский государственный технический университет**  
**имени Н.Э. Баумана**  
**(национальный исследовательский университет)»**  
**(МГТУ им. Н.Э. Баумана)**

---

ФАКУЛЬТЕТ \_\_\_\_\_ «Информатика и системы управления»

КАФЕДРА \_\_\_\_\_ «Теоретическая информатика и компьютерные технологии»

**Лабораторная работа № 5**  
**по курсу «Алгоритмы компьютерной графики»**  
**«Алгоритмы отсечения»**

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

*Москва 2025*

# 1 Задача

а. Реализовать один из алгоритмов отсечения определенного типа в пространстве заданной размерности (в соответствии с вариантом).

б. Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши.

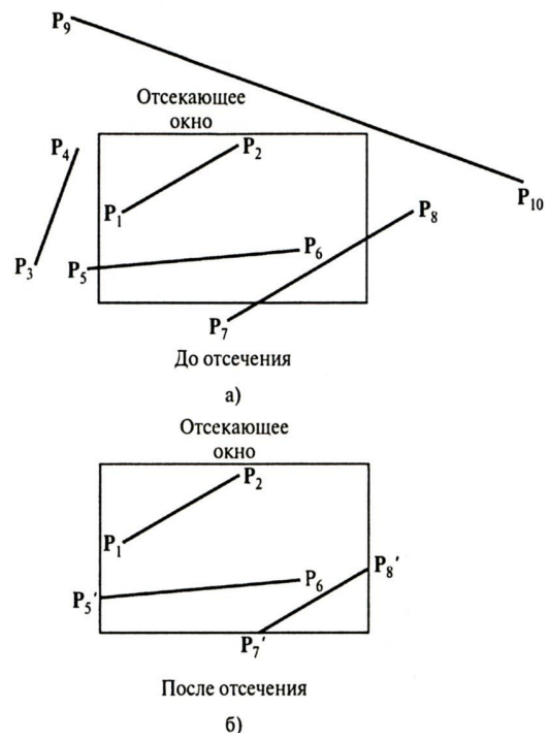
Персональный вариант:

- Алгоритм отсечения: отрезка произвольным многоугольником
- Размерность пространства отсечения: Трехмерное
- Тип отсечения: Внутреннее

## 2 Основная теория

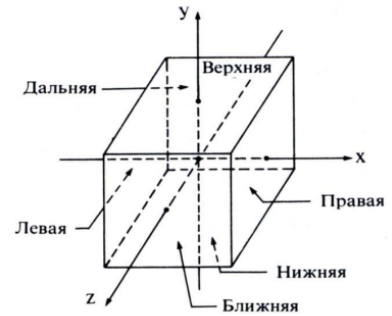
### Отсечение

- *алгоритмом отсечения (отсечением)* называется любая процедура, которая удаляет те точки изображения, которые находятся внутри (или снаружи) заданной области пространства;
- отсекающее окно или объем называются *отсекателем*:
  - регулярной формы (прямоугольное окно или параллелепипед, стороны которого параллельны осям координат);
  - нерегулярной формы (выпуклый или невыпуклый многоугольник или многогранник).



# Классификация алгоритмов отсечения

- **по типу обрабатываемых объектов:**
  - отсечение точки;
  - отсечение линии (отрезка);
  - отсечение области (многоугольника);
  - отсечение кривой;
  - отсечение текста (литер);
- **по размерности:**
  - двумерное отсечение;
  - трехмерное отсечение;
- **по расположению результата отсечения относительно отсекателя:**
  - внутреннее отсечение;
  - внешнее отсечение.



а. Параллелепипед



б. Усеченная пирамида

$$xw_{\min} \leq x \leq xw_{\max},$$

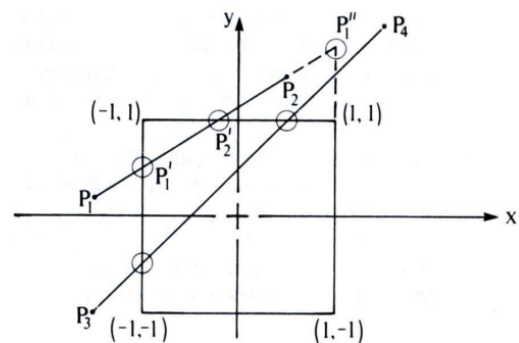
$$yw_{\min} \leq y \leq yw_{\max}.$$

## Простой алгоритм отсечения отрезка

- основан на отсечении точки;
- реализует двумерное отсечение отрезка регулярным окном;
- для каждой из сторон отсекателя:
  - рассчитывается точка пересечения прямой, содержащей сторону отсекателя, с прямой, содержащей отрезок;
  - анализируется принадлежность точки пересечения области отсекателя;
- недостаток: определяются 4 точки пересечения;
- вычисление точки пересечения является ресурсоемкой операцией, количество которых необходимо минимизировать.

$$xw_{\min} \leq x \leq xw_{\max},$$

$$yw_{\min} \leq y \leq yw_{\max}.$$



$$\text{с левой: } x_L, y = m(x_L - x_1) + y_1 \quad m \neq \infty$$

$$\text{с правой: } x_P, y = m(x_P - x_1) + y_1 \quad m \neq \infty$$

$$\text{с верхней: } y_B, x = x_1 + (1/m)(y_B - y_1) \quad m \neq 0$$

$$\text{с нижней: } y_H, x = x_1 + (1/m)(y_H - y_1) \quad m \neq 0$$

# Определение положения точки относительно прямой (плоскости)

- через уравнение (неявное задание) ориентированной прямой:

$$F(x,y)=(y_2-y_1)*(x-x_1)-(x_2-x_1)*(y-y_1)=0$$

- если  $F(x_3,y_3)=0$ , то точка  $O(x_3,y_3)$  лежит на прямой;
- если  $F(x_3,y_3)>0$ , то точка  $O(x_3,y_3)$  расположена справа от прямой;
- если  $F(x_3,y_3)<0$ , то точка  $O(x_3,y_3)$  расположена слева от прямой;

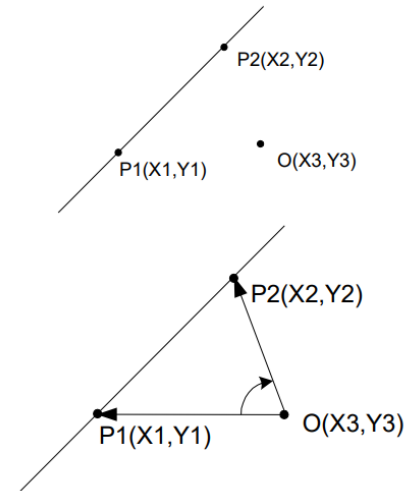
- через скалярное произведение нормали к отрезку и вектора, направленного из произвольной точки на отрезке в тестируемую точку (анализируется знак скалярного произведения):

$$\mathbf{N} = (y_2 - y_1, -(x_2 - x_1))$$

$$F(x_3, y_3) = (\mathbf{N}, \mathbf{P}_1 \mathbf{O})$$

- через векторное произведение векторов, направленных из тестируемой точки в вершины отрезка (анализируется знак координаты z векторного произведения).

$$OP_1 \times OP_2 = \begin{vmatrix} X_1 - X_3 & Y_1 - Y_3 & 0 \\ X_2 - X_3 & Y_2 - Y_3 & 0 \\ i & j & k \end{vmatrix}$$



## Пересечение прямых и плоскостей

- пересечение двух отрезков прямых:

$$A + bt = C + du, \quad t = \frac{\mathbf{d}^\perp \cdot \mathbf{c}}{\mathbf{d}^\perp \cdot \mathbf{b}}, \quad u = \frac{\mathbf{b}^\perp \cdot \mathbf{c}}{\mathbf{d}^\perp \cdot \mathbf{b}},$$

$$bt = \mathbf{c} + du, \quad I = A + \left( \frac{\mathbf{d}^\perp \cdot \mathbf{c}}{\mathbf{d}^\perp \cdot \mathbf{b}} \right) \mathbf{b} \text{ (точка пересечения).}$$

» для отрезков производится проверка  $t, u \in [0, 1]$

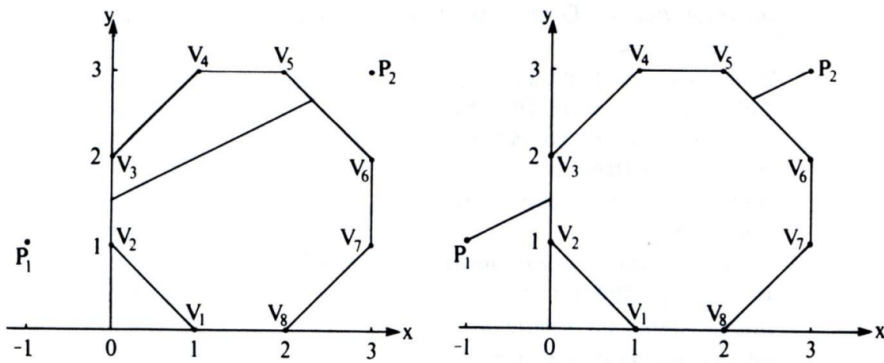
- пересечение прямой и плоскости:

$$\mathbf{n}(A + ct_{hit} - B) = 0, \quad t_{hit} = \frac{\mathbf{n}(B - A)}{\mathbf{n} \cdot \mathbf{c}},$$

$$\mathbf{n}(A - B) + \mathbf{n} \cdot \mathbf{c} t_{hit} = 0,$$

» для отрезка производится проверка  $t \in [0, 1]$

## Внутреннее и внешнее отсечение



- внутреннее отсечение:  $t \in [t_H; t_B]$ ;
- внешнее отсечение:  $t \in [0; t_H] \cup [t_B; 1]$ .

## 3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLU import *
4 import numpy as np
5
6 cube_scale = 1.0
7 rotation_x = 0.0
8 rotation_y = 0.0
9 line_start = [-2, 0, 0]
10 line_end = [2, 0, 0]
11
12 def get_cube_faces(scale):
13     s = scale
14     faces = [
15         ([ s, 0, 0], [-1, 0, 0]),
16         ([-s, 0, 0], [ 1, 0, 0]),
17         ([0,  s, 0], [ 0, -1, 0]),
18         ([0, -s, 0], [ 0, 1, 0]),
19         ([0, 0,  s], [ 0, 0, -1]),
20         ([0, 0, -s], [ 0, 0, 1]),
21     ]
22     return faces
23
24 def intersect_line_plane(p1, p2, plane_point, plane_normal):
```

```

25     p1 = np.array(p1, dtype=np.float64)
26     p2 = np.array(p2, dtype=np.float64)
27     plane_point = np.array(plane_point, dtype=np.float64)
28     plane_normal = np.array(plane_normal, dtype=np.float64)
29
30     d = p2 - p1
31     denom = int(np.dot(plane_normal, d))
32
33     if abs(denom) < 1e-8:
34         return None
35
36     t = int(np.dot(plane_normal, plane_point - p1)) / denom
37
38     if 0 <= t <= 1:
39         return p1 + t * d
40     else:
41         return None
42
43 def point_inside_polyhedron(point, faces):
44     point = np.array(point, dtype=np.float64)
45     for plane_point, normal in faces:
46         plane_point = np.array(plane_point, dtype=np.float64)
47         normal = np.array(normal, dtype=np.float64)
48         if np.dot(normal, point - plane_point) < -1e-8:
49             return False
50     return True
51
52 def clip_line_simple(p1, p2, faces):
53     p1 = np.array(p1, dtype=np.float64)
54     p2 = np.array(p2, dtype=np.float64)
55     points = []
56
57     if point_inside_polyhedron(p1, faces):
58         points.append(p1)
59     if point_inside_polyhedron(p2, faces):
60         points.append(p2)
61
62     for plane_point, normal in faces:
63         pt = intersect_line_plane(p1, p2, plane_point, normal)
64         if pt is not None and point_inside_polyhedron(pt, faces):
65             if not any(np.allclose(pt, existing, atol=1e-6) for existing
66             in points):
67                 points.append(pt)
68
69     if len(points) < 2:
70         return None

```

```

70
71     points = sorted(points, key=lambda pt: np.linalg.norm(pt - p1))
72     return points[0].tolist(), points[1].tolist()
73
74 def draw_cube(scale):
75     s = scale
76     glBegin(GL_LINES)
77     glColor3f(0.5, 0.5, 0.5)
78     edges = [
79         [-s, -s, -s], [ s, -s, -s],
80         [-s,  s, -s], [ s,  s, -s],
81         [-s, -s,  s], [ s, -s,  s],
82         [-s,  s,  s], [ s,  s,  s],
83         [-s, -s, -s], [-s,  s, -s],
84         [ s, -s, -s], [ s,  s, -s],
85         [-s, -s,  s], [-s,  s,  s],
86         [ s, -s,  s], [ s,  s,  s],
87         [-s, -s, -s], [-s, -s,  s],
88         [ s, -s, -s], [ s, -s,  s],
89         [-s,  s, -s], [-s,  s,  s],
90         [ s,  s, -s], [ s,  s,  s],
91     ]
92     for i in range(0, len(edges), 2):
93         glVertex3fv(edges[i])
94         glVertex3fv(edges[i + 1])
95     glEnd()
96
97 def draw_line(p0, p1, color, width = 1.0):
98     glLineWidth(width)
99     glColor3f(*color)
100    glBegin(GL_LINES)
101    glVertex3fv(p0)
102    glVertex3fv(p1)
103    glEnd()
104    glLineWidth(1.0)
105
106 def input_line():
107     try:
108         print("Begin line (x y z):")
109         x1, y1, z1 = map(float, input(">>> ").split())
110         print("End line (x y z):")
111         x2, y2, z2 = map(float, input(">>> ").split())
112         return [x1, y1, z1], [x2, y2, z2]
113     except Exception as e:
114         print(f"Error input: {e}")
115     return None, None

```

```

116
117 def key_callback(window, key, scancode, action, mods):
118     global cube_scale, rotation_x, rotation_y
119     if action == glfw.PRESS or action == glfw.REPEAT:
120         if key == glfw.KEY_EQUAL or key == glfw.KEY_KP_ADD:
121             cube_scale += 0.1
122         elif key == glfw.KEY_MINUS or key == glfw.KEY_KP_SUBTRACT:
123             cube_scale = max(0.1, cube_scale - 0.1)
124         elif key == glfw.KEY_S:
125             rotation_x -= 5.0
126         elif key == glfw.KEY_E:
127             rotation_x += 5.0
128         elif key == glfw.KEY_F:
129             rotation_y -= 5.0
130         elif key == glfw.KEY_D:
131             rotation_y += 5.0
132         elif key == glfw.KEY_L:
133             global line_start, line_end
134             glfw.iconify_window(window)
135             p0, p1 = input_line()
136             glfw.restore_window(window)
137             if p0 and p1:
138                 line_start, line_end = p0, p1
139
140 def main():
141     global rotation_x, rotation_y
142
143     if not glfw.init():
144         return
145
146     window = glfw.create_window(1000, 750, "Lab5", None, None)
147     if not window:
148         glfw.terminate()
149         return
150
151     glfw.make_context_current(window)
152     glfw.set_key_callback(window, key_callback)
153
154     glEnable(GL_DEPTH_TEST)
155
156     while not glfw.window_should_close(window):
157         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
158         glMatrixMode(GL_PROJECTION)
159         glLoadIdentity()
160         gluPerspective(45, 800 / 600, 0.1, 100)
161         glMatrixMode(GL_MODELVIEW)

```



```

162         glLoadIdentity()
163
164         gluLookAt(0, 0, 5, 0, 0, 0, 0, 1, 0)
165
166         glRotatef(rotation_x, 1, 0, 0)
167         glRotatef(rotation_y, 0, 1, 0)
168
169         draw_cube(cube_scale)
170
171         draw_line(line_start, line_end, (0.6, 0.6, 0.6), width=3.0)
172
173         clipped = clip_line_simple(np.array(line_start), np.array(
line_end), get_cube_faces(cube_scale))
174         #print(clipped)
175         if clipped:
176             glDisable(GL_DEPTH_TEST)
177             draw_line(clipped[0], clipped[1], (1, 0, 0), width=5.0)
178             glEnable(GL_DEPTH_TEST)
179
180         glfw.swap_buffers(window)
181         glfw.poll_events()
182
183     glfw.terminate()
184
185 main()

```

Результат запуска представлен на рисунке 1.

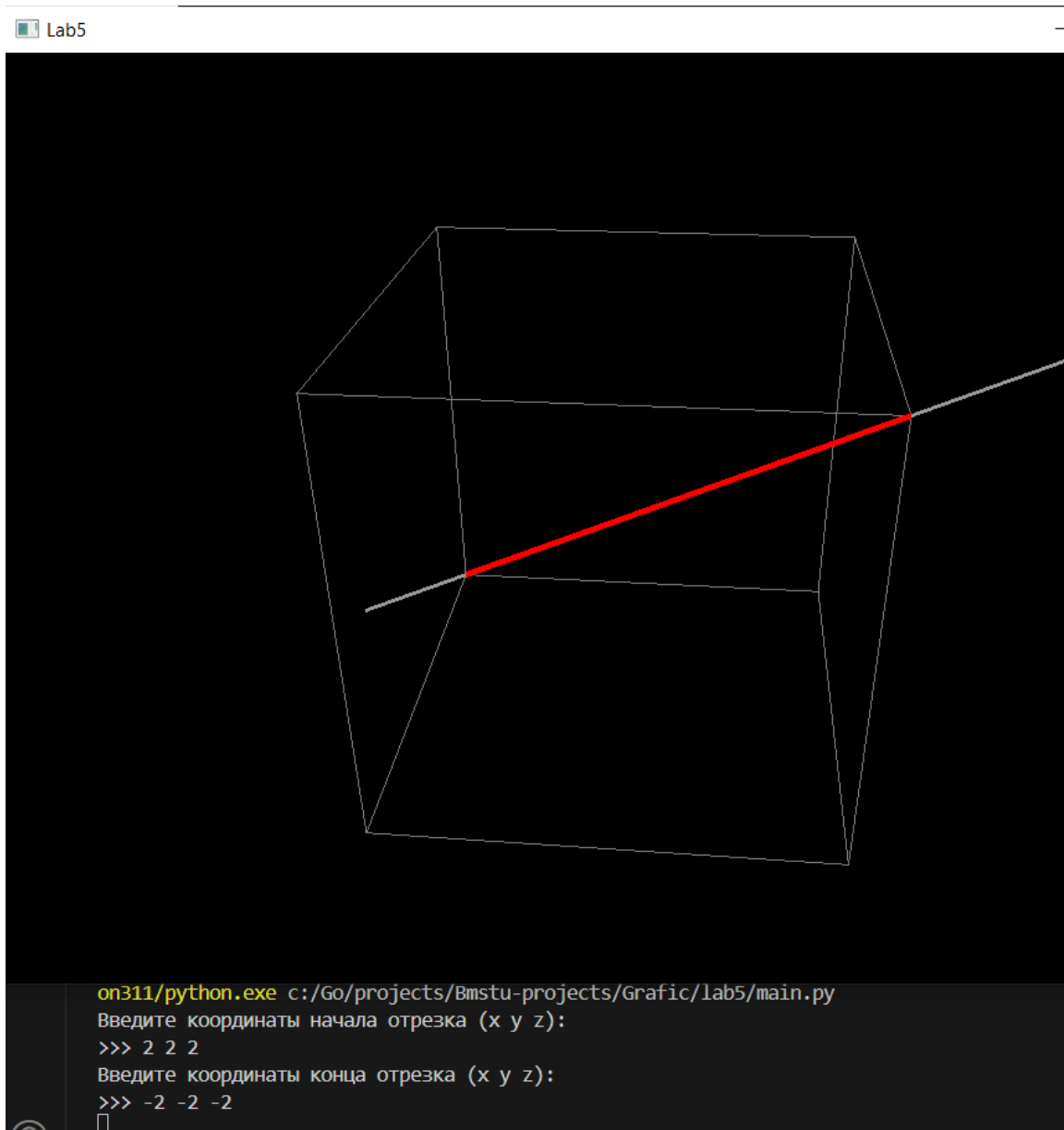


Рис. 1 — Результат с отрезком с началом  $[2, 2, 2]$  и концом  $[-2, -2, -2]$

## 4 Заключение

В рамках лабораторной работы была реализована программа, выполняющая внутреннее отсечение отрезка произвольным выпуклым многогранником в трёхмерном пространстве. Пользователю предоставляется возможность вводить координаты отрезка вручную с клавиатуры. В качестве отсекателя используется куб. Реализация отсечения основана на простом алгоритме: производится пересечение отрезка с плоскостями всех граней, после чего анализируется принадлежность полученных точек отсекателю. Визуализация сцены осуществляется с использованием библиотеки OpenGL, обеспечивающей интерактивное

отображение и управление фигурой. Отображение отрезка и его отсечённой части реализовано с различной толщиной и цветом для наглядности. Результат демонстрирует корректную работу алгоритма во всех предусмотренных вариантах расположения отрезка по отношению к отсекателю.